

YAPar

CC3071 – Diseño de Lenguajes de Programación
Camila Richter Marinés García Jose Antonio Mérida
github.com/Jose-Merida-UVG/Compilers-1-CC3071
19 de mayo de 2026

Diseño de Software

Descripción general

YAPar (Yet Another Parser) es una herramienta de generación de analizadores sintácticos inspirada en Yacc y ocaml yacc. A partir de un archivo de especificación `.yalp` y un archivo léxico `.yal` procesado por YALex, el sistema construye el autómata LR(0), calcula los conjuntos FIRST y FOLLOW, construye la tabla SLR(1), y genera un programa Go completo que implementa el analizador. El proyecto fue desarrollado en Go (backend) y React + TypeScript (frontend), sin librerías de parsing externas. Documentación detallada del código se encuentra en el repositorio.

Arquitectura

El sistema se divide en dos capas: el servidor HTTP en Go que expone una API REST y ejecuta el pipeline de compilación, y la interfaz en React que permite al usuario interactuar con el generador.

Módulo	Responsabilidad
<code>internal/yalex</code>	Parseo del <code>.yal</code> , expansión de let-defs, pipeline léxico completo.
<code>internal/yapar</code>	Parseo del <code>.yalp</code> , gramática, FIRST/FOLLOW, LR(0), SLR(1), validación de tokens.
<code>internal/codegen</code>	Generación de <code>lexer.go</code> y <code>parser.go</code> .
<code>internal/graph</code>	Serialización de DFA y autómata LR(0) a JSON para el frontend.
<code>internal/ds</code>	Stack y TreeNode genéricos.
Frontend	Editor Monaco, visores DFA/LR(0)/SLR(1) con Graphviz WASM, terminal.

Pipeline Técnico

El proceso completo desde los archivos de especificación hasta el parser ejecutable se resume en las siguientes etapas:

1. **Parseo:** `ParseYalContent` y `ParseYalpContent` construyen las estructuras internas de ambos archivos. `ValidateTokenCoverage` cruza los conjuntos de tokens, asegurando que

cada `%token` tenga su `return` en el lexer y viceversa.

2. **Pipeline léxico:** Para cada patrón del `.yal` se ejecuta normalización, concatenación explícita, shunting-yard, construcción del AST con followpos, y construcción directa del DFA. Cada DFA se minimiza con Hopcroft. Los DFAs se fusionan en uno solo mediante Product Construction y se minimizan nuevamente.
3. **Gramática:** `grammar.Build` clasifica los símbolos de las producciones y construye la `Grammar`. `Augment()` antepone $S' \rightarrow S$ en el índice cero.
4. **FIRST y FOLLOW:** Calculados sobre la gramática aumentada mediante iteración hasta punto fijo.
5. **Autómata LR(0):** `lr0.BuildAutomaton` construye la colección canónica de conjuntos de ítems mediante clausura y goto.
6. **Tabla SLR(1):** `slr.Build` llena las tablas ACTION y GOTO. Los conflictos se registran sin interrumpir la construcción.
7. **Generación de código:** `GenerateCombined` y `GenerateParser` producen `lexer.go` y `parser.go`, formando un único package `main` ejecutable.

Algoritmos Principales

Conjuntos FIRST y FOLLOW

Ambos conjuntos se construyen con *fixed-point iteration*: se recorren todas las producciones acumulando terminales en los conjuntos de cada no-terminal, y se repite hasta que ninguna pasada agrega nada nuevo. La nulabilidad — si un símbolo puede derivar ε — se almacena como $\varepsilon \in \text{FIRST}(A)$ y se propaga en ambos cálculos.

Para FIRST: dado $A \rightarrow X_1X_2 \dots X_n$, se agrega $\text{FIRST}(X_1)$ a $\text{FIRST}(A)$; si X_1 es nutable se continúa con X_2 , y así sucesivamente. Para FOLLOW: se inicializa $\text{FOLLOW}(S') = \{\$ \}$. Para cada producción $A \rightarrow \alpha B \beta$, se agrega $\text{FIRST}(\beta) \setminus \{\varepsilon\}$ a $\text{FOLLOW}(B)$; si β es completamente nutable, se agrega también $\text{FOLLOW}(A)$. Ambos procesos iteran hasta *convergencia*.

Autómata LR(0)

Un ítem LR(0) es un par (*índice de producción, posición del punto*). El punto indica cuánto del cuerpo se ha reconocido; un ítem completo tiene el punto al final.

Para la **clausura**: dado un conjunto de ítems, para cada ítem donde el punto precede a un no-terminal B , se agregan todos los ítems de la forma $B \rightarrow \bullet \gamma$ al conjunto. Esto se repite hasta que no se agregan ítems nuevos.

Para **goto**(I, X): filtra los ítems de I donde el símbolo tras el punto es X , avanza el punto una posición en cada uno, y devuelve la clausura del resultado.

La **construcción del autómata** en `BuildAutomaton` sí usa un *worklist*: se parte del estado

inicial (clausura del ítem $S' \rightarrow \bullet S$) y se itera sobre un slice de estados que crece dinámicamente. Para cada estado se recolectan los símbolos que aparecen después de algún punto, se llama a goto para cada uno, y si el resultado es un estado nuevo se agrega al slice. Los estados se identifican por una clave canónica sobre sus ítems para detectar duplicados.

Tabla SLR(1)

Para cada estado y cada ítem en él, se llena la tabla así:

- Punto antes de un terminal $a \Rightarrow \text{SHIFT}$ al estado indicado por goto.
- Punto antes de un no-terminal $B \Rightarrow \text{llenar GOTO}[s][B]$.
- Ítem completo $A \rightarrow \alpha \bullet$, producción aumentada $\Rightarrow \text{ACCEPT}$ en \$.
- Ítem completo $A \rightarrow \alpha \bullet$, cualquier otra producción $\Rightarrow \text{REDUCE}$ en cada $t \in \text{FOLLOW}(A)$.

Si una celda ya tiene un valor diferente al que se intenta escribir, se registra un *conflict* (shift/reduce o reduce/reduce) sin interrumpir la construcción. Luego, se envía un mensaje de error informando que la gramática no es SLR(1) y las producciones que generan este conflicto.

Archivos de Prueba

Los siguientes pares `.yal/.yalp` se encuentran en `workspace/specs/` y tienen archivos de entrada en `workspace/input/`:

- **arithmetic** – expresiones aritméticas con precedencia completa codificada en la jerarquía de no-terminales: `expr > term > power > atom`. Soporta literales enteros y flotantes, identificadores, la función built-in `pow(a,b)`, y llamadas a función con lista de argumentos.
- **calc** – lenguaje imperativo con declaraciones `let`, asignaciones simples y compuestas (`+=`, `-=`, `*=`, `/=`), condicionales `if/else`, ciclos `while`, y bloques delimitados por paréntesis. Usa una jerarquía `expr` unificada que cubre aritmética y lógica con precedencia completa: OR, AND, NOT, comparación, suma, multiplicación, potencia (`**`), y unario. Requiere **rebuild** desde el frontend.
- **gol** – subconjunto de Go. Soporta declaración de paquete, imports, funciones con parámetros y tipo de retorno, `var`, `const`, bloques, sentencias de asignación y definición corta (`:=`), incremento/decremento, `if/else`, `for`, `return`, y expresiones con precedencia completa incluyendo acceso a campos, indexación y llamadas (*postfix*). El archivo de prueba implementa la función Fibonacci iterativa. Verificado: **acepta correctamente**.

Los siguientes se encuentran en `workspace/demo/` y sirven para verificar el visualizador de autómatas LR(0) y los conjuntos FIRST/FOLLOW:

- **dragon** – gramática clásica de expresiones del Dragon Book §4.5: $e \rightarrow e + t \mid t, t \rightarrow t * f \mid f, f \rightarrow (e) \mid \text{ID}$. Gramática no ambigua canónica, útil para verificar que los conjuntos FIRST/FOLLOW y el autómata LR(0) coincidan con los valores del libro.

- **epsilon_test** – gramática con múltiples producciones épsilon anidadas, diseñada para verificar la propagación correcta de nulabilidad en el cómputo de FIRST y FOLLOW. Esta fue utilizada como ejercicio en clase, se utilizó para comparar los cálculos FIRST y FOLLOW del programa con los resultados teóricos a mano.

Anexos

Repositorio GitHub (rama `parser`):

<https://github.com/Jose-Merida-UVG/Compilers-1-CC3071/tree/parser>